

ELEC 391: Cascaded PID Control of a Two-Wheel Inverted Pendulum Robot

April 13, 2025

1st Mazda Farrahi
Electrical and Computer Engineering
The University of British Columbia
Vancouver, Canada
mazda.farrahi@gmail.com

2nd Qinnan Zhou
Electrical and Computer Engineering
The University of British Columbia
Vancouver, Canada
alexzhou330@gmail.com

CONTENTS		
I	Introduction	2
II	Project Specifications	2
III	Physical & Hardware Design	2
III-A	Physical Overview	2
III-B	Hardware Overview	3
IV	System Modeling	4
IV-A	System Representation	4
IV-B	System State Space	6
IV-C	Measuring System Parameters	7
IV-C1	Experimentally Determining Centers of Mass	7
IV-C2	Experimentally Determining Moments of Inertia	7
IV-D	PID Design	7
IV-E	Simulation Considerations in MATLAB	7
IV-E1	Experimentally Determining G_a	7
IV-E2	Experimentally Determining Y_{elec}	8
IV-F	Simulink Modeling	8
V	Software Design	8
V-A	IMU Pitch Angle Reading	8
V-B	Magnetic Encoders	9
V-C	PID Loop	9
V-D	GUI and Bluetooth Communication	9
V-E	Extra Features: ESP32 Cam and Steering Wheel	10
VI	Verification and Validation	10
VI-A	Simulation Results	10
VI-B	Real-World Results	10
VII	Conclusion	10
VII-A	Fulfillment of Goals	10
VII-B	Future Recommendations	11
Appendix A: Budget		11
Appendix B: Technical Calculations and Simulations		11
B-A	Non-linear Systems Equations Derivation using Lagrangian Mechanics	11
B-B	Derivation of the Linear State-Space System	12
Appendix C: Schematics		13
Appendix D: Datasheet and URLs for Selected Components		13

Appendix E: Prototypes	13
Appendix F: Sample Code	13
F-A Python Functions to find Mo- ment of Inertia	13
Appendix G: Additional Figures	13
References	14

Abstract—This project report paper details the entire process of implementing a self-balancing, maneuverable two-wheeled inverted pendulum robot. Main aspects include system modeling, control simulation, and careful considerations pertaining to microcontroller programming, PID setup, and system model validation.

Index Terms—controls, systems, modeling, programming, PID.

I. INTRODUCTION

The classical inverted pendulum problem consists of a mechanism with a rotating joint and a center of mass above the joint, which inherently results in an unbalanced system. This system can be represented by a pendulum attached to a moving cart by a rotating joint, which can be balanced by precisely moving the cart to align the center of mass with the axis of rotation.

In this course, we have been tasked with creating a two-wheeled inverted pendulum vehicle (Fig. 1), which extends the classic inverted pendulum controls problem. This project involves extensive MATLAB simulation and modeling of the physical system and its PID systems, as well as software and hardware design/integration to implement a fully functional prototype of the robot with maneuverability over Bluetooth.

For this project, we used an Arduino Nano Sense rev-2 BLE as our MCU (micro-controller unit), and its onboard IMU's (inertial measurement unit) accelerometer and gyroscope to obtain the tilt angle of the robot. Furthermore, we integrated magnetic encoders on the robot's wheels, which allowed for high-accuracy readings of the robot's displacement and yaw angle. Our extra features include a GUI (graphical user interface) for streaming video via an ESP32 camera module, along with controlling and

tuning the robot over Bluetooth. We also created a custom motion sensing steering wheel, allowing for greater comfort and precision when operating the robot.

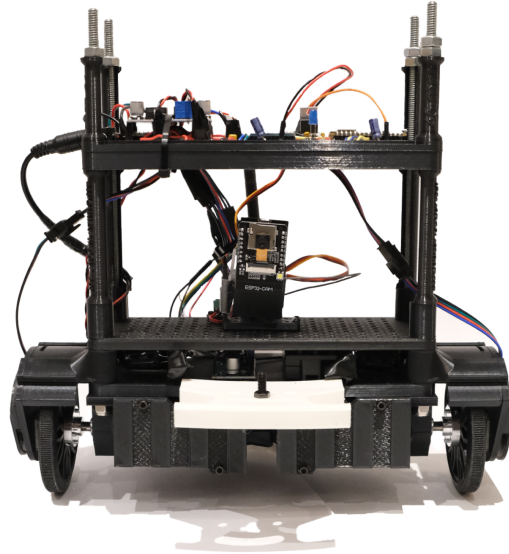


Fig. 1. Front view of our final robot design.

II. PROJECT SPECIFICATIONS

The project requirements can be summarized in Tables IV, II, and III. Most of the outlined requirements are defined by the course instructors as base requirements. However, we must also create our own requirements for our desired extra features. From the tables, these are the items involving mentions of “video” or “camera”.

III. PHYSICAL & HARDWARE DESIGN

A. Physical Overview

There are a few notable improvements between our final design (Fig. 1) and the original kit (Fig. 15, Appendix G). The main changes and their respective reasons are as follows:

- **Encoder Mount and Magnet Wheel Mount** (Fig. 18, Appendix G): Houses the AS5600

TABLE I
FUNCTIONAL REQUIREMENTS

<i>Functional Requirement</i>	<i>Description</i>
Balancing	Able to balance at a vertical equilibrium position
Speed control	Balancing while being driven forward and backward
Direction control	Balancing while taking turns statically
Bluetooth	Controlled wirelessly from the GUI by a BLE signal with 4 directions; forward, backward, left, right
Video	Control the robot using only the camera feed on the GUI
Camera Stabilization	Stabilize the robot's camera using a servo to always stay level with the horizon
Steering Wheel	Fully control the robot using the steering wheel's buttons and orientation (IMU)

TABLE II
PERFORMANCE REQUIREMENTS

<i>Performance Requirement</i>	<i>Description</i>
Balancing Disturbance	Able to recover from a 15° disturbance
Speed	Balancing while moving at a speed of 0.35m/s
Rotation	Balancing while rotating at an angular velocity of 30°/s
Controls error/overshoot	Less than 10cm of drift when vertically balanced
Video stream	≥20fps, minimal lag and stabilized video allowing for responsive FPV control

magnetic encoder reader, which reads the rotating magnet fixed on the wheel.

- **Lowered 8×AA Battery Pack** (Fig. 19, Appendix G): Improves model simulation accuracy. Considering that the batteries are quite heavy and thus has a large inertia, our previous battery-pack was unable to consistently stay fixed during rapid movement. This new design not only fixes the center-of-mass in place, but lowers it as well. This new center-of-mass location better aligns with how the system model

TABLE III
CONSTRAINTS

<i>Constraint Requirement</i>	<i>Description</i>
Cost	\$65 (CAD) budget requirement (tax and shipping are omitted)
Resources	Must use the components (MCU, Motors, Drivers, etc) in the ELEC 391 robot kit. The power source is also limited to the provided battery pack.
Time	The project must meet the presentation deadline on April 8th, 2025, about 10 weeks to complete the project after given the robot kit

(Fig. 3) assumes the robot's lower portion, M , to be inline with the axle.

- **Breadboard Chassis** (Fig. 17, Appendix G): Holding the breadboard in-place maintains a fixed position for the IMU on the Arduino, relative to the robot body. This eliminates any inaccuracies in the IMU readings due to physical variability.

To produce the required physical components, we first created their CAD (Computer Aided Design) models using Autodesk Fusion 360 which we 3D-printed using PLA plastic.

B. Hardware Overview

The primary MCU for this robot is an Arduino Nano BLE 33 Sense Rev2, which provides Bluetooth functionality and comes with an onboard BMI270 IMU. The BMI270 is a 6-axis accelerometer and gyroscope with 16-bit resolution, which is essential to inverted pendulum projects for determining the tilt angle. There are two AS5600 magnetic encoders mounted by the wheels, allowing us to read the angular position of the wheels, which we could use in the software to calculate the displacement and the yaw angle of the robot. AS5600 magnetic encoders have the same I2C address, therefore, a TCA9548A I2C multiplexer is required for communication with the

Arduino.

For the robot's movement, we were provided the Pololu 4741 DC motors, driven by the DRV8833 dual H-bridge. Each motor was driven by two PWM signals, with their duty cycle (between 0-255) setting the motor's forward and backward speed, respectively. Through testing the motors' deadzone (the smallest PWM duty cycle required to generate rotation) and their deceleration rate, we found that the slow decay method, where all PWM signals begin at 255, resulted in the fastest deceleration and the smallest deadzone, allowing for more responsive control of the motors.

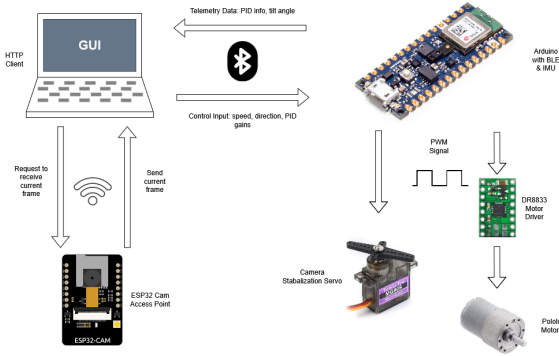


Fig. 2. Overview of the hardware components and their interactions.

For our extra features, we used an ESP32 Camera module set up as a central access point mounted to an MG90S servo motor for active stabilization. The steering wheel was fully 3D printed out of PLA and TPU, and incorporated push buttons for forward and backward control, an MPU6050 IMU measuring the rotation about x-axis for direction control, and two paddle shifter switches allowing for the change of gears. We chose to use an Arduino UNO as the MCU for the steering wheel due to its ease of use with the IMU, built-in pull-up resistors for reading pushbuttons and reliability.

The ESP32 Camera can consume up to 310mA of current, therefore we chose to use two buck

converters, one for the ESP32 cam, and one for the Arduino and the servo and the I2C components, to efficiently step down the battery pack's 12V to 10V while supplying the required current. Through experimentation, we also found that having three 2200 μ F decoupling capacitors in parallel with each buck converter's ground, reduced the noise in the circuit and prevented back EMF from turning off the Arduino Nano.

IV. SYSTEM MODELING

A. System Representation

We began by separating our robot system into two components: the heavy base that include the batteries and motors, and the lighter upper layers containing the MCU. This allows us to represent the system as a classic 2DOF (2 degrees of freedom) "inverted pendulum-on-a-cart" problem. The base of the robot is the "cart", and the upper-levels act as the "pendulum" mass. For this simplified model, we initially considered two motions: x - the linear movement of the cart (base), and θ - the tilt angle with the normal to ground being 0° (also known as the pitch angle). [1].

The states of our system are:

$$\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}, \delta, \dot{\delta}]^T \quad (1)$$

Where the state variables represent:

- x Position.
- θ Pitch Angle.
- δ Yaw Angle.

It is shown in [1] that the state-space matrix representation (omitting frictional forces) for the **A** and **B** matrices can be represented as:

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \\ \dot{\delta} \\ \ddot{\delta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & A_{23} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & A_{43} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \\ \delta \\ \dot{\delta} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ B_{21} & B_{22} \\ 0 & 0 \\ B_{41} & B_{42} \\ 0 & 0 \\ B_{61} & B_{62} \end{bmatrix} \begin{bmatrix} \tau_L \\ \tau_R \end{bmatrix} \quad (2)$$

Where A_{23} , A_{43} , B_{21} , B_{22} , B_{41} , B_{42} , B_{61} , B_{62} are derived from the system's physical parameters. By observing that the $\mathbf{A}$ matrix from 2 can be decomposed into two sub-systems; one for x - θ (Eq. 3) and the other for δ (Eq. 4), we can simplify our model greatly.

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & A_{23} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & A_{43} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ B_{21} & B_{22} \\ 0 & 0 \\ B_{41} & B_{42} \end{bmatrix} \begin{bmatrix} \tau_L \\ \tau_R \end{bmatrix} \quad (3)$$

$$\begin{bmatrix} \dot{\delta} \\ \ddot{\delta} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \delta \\ \dot{\delta} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ B_{61} & B_{62} \end{bmatrix} \begin{bmatrix} \tau_L \\ \tau_R \end{bmatrix} \quad (4)$$

Graphically, these two sub-systems are shown in Fig. 3 and 5 for x - θ , and Fig. 4 for δ .

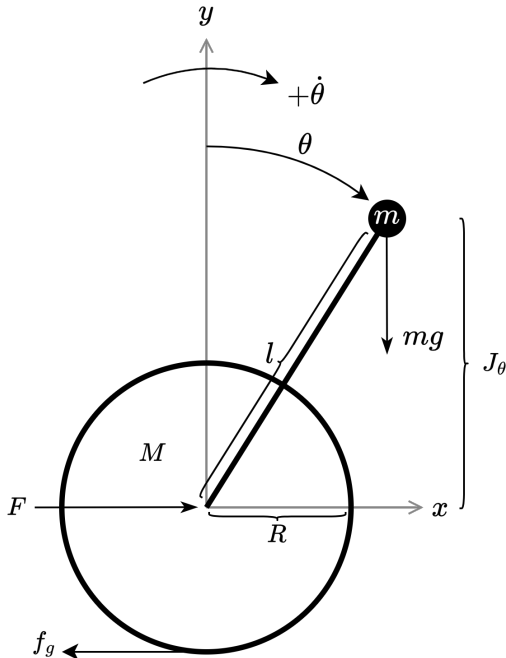


Fig. 3. Free body diagram for the side view of the robot.

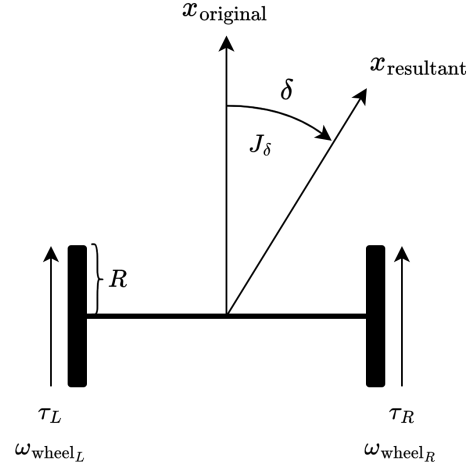


Fig. 4. Free body diagram of the top-down view, capturing δ (Yaw) dynamics of the robot.

The parameters from Fig. 3 and 4 are described as follows:

- F Input force.
- τ Input torque (for the x - θ sub-system).
- τ_L Torque from the left wheel.
- τ_R Torque from the right wheel.
- m Robot's upper mass (pendulum).
- M Robot's lower mass (cart).
- l Distance between the upper and lower masses. (The lower mass's center of gravity is assumed to lie on the axle.)
- R Radius of the wheel.
- g Earth's gravitational constant.
- f_g Friction from ground.
- J_θ Pitch moment of inertia.
- J_δ Yaw moment of inertia.

Fig. 5 provides a more detailed diagram describing the physics acting on the wheels and lower portion [1]. Since we are considering the pendulum physics relative to the lower chassis, the normal force F_N is what m “feels” in its interaction with M . Similarly, the reaction force F_R is felt by the lower chassis M in response to interacting with m .

- T_N Normal “tension” force.

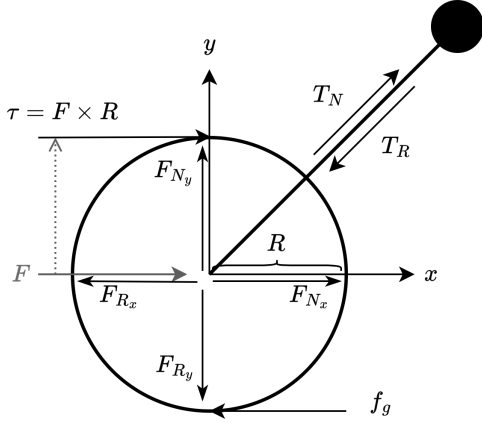


Fig. 5. Free body diagram describing in greater detail, the dynamics and physics relative to the wheel/lower robot chassis, M .

- T_R Reaction “tension” force.
- F_{N_x} Normal force’s horizontal component.
- F_{N_y} Normal force’s vertical component.
- F_{R_x} Reaction force’s horizontal component.
- F_{R_y} Reaction force’s vertical component.

By using normal and reaction forces, it becomes intuitive in understanding how torque from the wheels translate into balancing the upper mass. With an input torque applied at the wheels, it is as-if we are applying an input force F at the wheel’s contact point with the ground (while considering the factor of R). By moving F to the wheel’s mid-point (axle), we can now use our “inverted pendulum-on-a-cart” analogy for our system. Assuming a tilt in the direction of x , a positive F increases F_{N_x} and F_{R_x} . The larger F_{R_x} causes T_R to “pull” mass m inwards, closer in position relative to the lower chassis M . Hence, through specific maneuvers from the wheel, stationary balance is achieved when mass m lies directly above M .

B. System State Space

Before we begin modeling, we need to consider that; despite Eq. 3 and 4 show separated sub-systems for x - θ and δ , we also need to decouple the inputs τ_L , τ_R to fully leverage our simplified “inverted pendulum-on-a-cart” model. As demonstrated in [1], matrix $\mathbf{Q}$ in Eq. 5 can be used

to produce τ_L and τ_R given τ_θ and τ_δ , enabling independent control for θ (pitch) and δ (yaw) (Eq. 6) to be used by our controller.

$$\mathbf{Q} = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{bmatrix} \quad (5)$$

$$\begin{bmatrix} \tau_L \\ \tau_R \end{bmatrix} = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{bmatrix} \begin{bmatrix} \tau_\theta \\ \tau_\delta \end{bmatrix} \quad (6)$$

In our actual model, we are producing this decoupling effect at the controller voltage output stage (PWM signal) instead of at torque, which follows the way we have programmed our robot. For modeling and simulation, Eq. 6 is a key step towards deriving the x - θ sub-system shown in Eq. 7 and 8 (derivation shown in Appendix B-B).

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{mg}{M} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -\frac{(M+m)g}{Ml} & 0 \end{bmatrix}; \mathbf{x} = \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} \quad (7)$$

$$\mathbf{B} = \begin{bmatrix} 0 \\ 1 \\ \frac{M}{0} \\ 1 \\ -\frac{1}{Ml} \end{bmatrix}; \mathbf{u} = \frac{\tau_\theta}{R} \quad (8)$$

Alternatively, the system can be represented using the moment of inertia. From [2] [3], the equations of the systems (omitting friction and ignoring the mass of the wheels) can be defined as:

$$\ddot{x} = \frac{\tau_L/R + \tau_R/R - ml \cos \theta (mgl \sin \theta)/J_\theta}{M + m - m^2 l^2 \cos^2 \theta / J_\theta} \quad (9)$$

$$\ddot{\theta} = \frac{(M + m)mgl \sin \theta - ml \cos \theta (\tau_L/R + \tau_R/R)}{J_\theta(M + m) - m^2 l^2 \cos^2 \theta} \quad (10)$$

Linearizing Eq. 9 and 10 based on methods described in Appendix B-B, we can re-establish the state-space matrices. For our modeling purposes, we did not use this method due to reasons that will be discussed in later sections.

C. Measuring System Parameters

1) *Experimentally Determining Centers of Mass:* Using an off-the-shelf fish scale, we first measured the total mass - M_t and then weighed only the base M (motor, batteries, wheels, and base-plate). m is found through simply performing $m = M_t - M$. By setting up Eq. 11, we can find the height of m 's center of mass y_m by assuming M 's center of mass to be inline with the axle, and measuring for M_t 's center of mass via balancing the robot's side.

$$M_t \cdot y_{M_t} = M \cdot y_M + m \cdot y_m \quad (11)$$

2) *Experimentally Determining Moments of Inertia:* The moment of inertia about an axis of interest can be determined experimentally through suspending the robot from two strings and timing the oscillation period after starting from a non-resting point in rotation [4] [5]. A photograph capturing the physical setup of the experiment for finding J_δ is shown in Fig. 20, Appendix G. The main equation for calculating the moment of inertia is given in Eq. 12, where d is the distance between the suspension ropes, and h is the height (length) of the ropes' hanging portion.

$$J = \frac{mgd^2T^2}{16\pi^2h} \quad (12)$$

To automate this process, a Python script was used. A snippet is located in Appendix F-A. It is important to note that J_θ was not used in the actual modeling due to the fact that the rotation from this test would be about the actual center of mass, whereas we desire J_θ to be about the axle. Hence, using J_θ would potentially cause inaccuracies that is best mitigated by using the alternative mass-based method.

D. PID Design

In our implementation, we cascaded the PIDs for controlling θ and x . This decision was based on the idea that, in order for the robot to move forward or backwards, it should tilt in the direction of travel and "catch" itself at its destination setpoint. To accomplish this, the θ -PID is placed after the x -PID (Fig. 6 and 12). When x_d (desired x) differs from x_a (actual x), the resulting x_e (error in x) causes

the x -PID to produce a non-zero value acting as θ_d (desired theta). This essentially "tilts" the robot in the desired direction as the θ -PID tries to fulfill θ_d . When the robot arrives at its destination, we would have $x_a = x_d$, meaning x_e is now 0 which also sets θ_d to be 0, effectively commanding the robot to solely balance at its location.

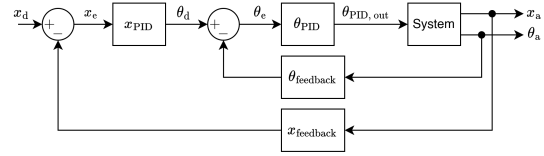


Fig. 6. Simplified block diagram of the x,θ -PID design.

E. Simulation Considerations in MATLAB

We considered factors such as actuator (motor driver) delay and gain, along with electrical-mechanical energy conversions. Fig. 7 shows a

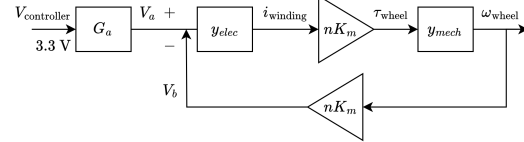


Fig. 7. Simplified Block Diagram of the Actuator & Electro-Mechanical Conversion Path.

common block diagram representing actuation and electro-mechanical conversion. The components are defined as:

- G_a Actuator (motor driver) dynamics.
- n Motor pinion to gear ratio.
- K_m Motor constant.
- Y_{elec} Electrical admittance.
- Y_{mech} Mechanical admittance.
- V_a Actuator voltage.
- V_b Motor back EMF.

n and K_m were identified from the official motor datasheet from Pololu. n was simply found to be 18.5, while K_m is found to be 16.129 mN·m/A.

1) *Experimentally Determining G_a :* To find G_a , we used a signal generator to produce square waves that were sent to the PWM inputs of motor driver.

These square waves span from 0 to 1V, which mimics a step signal. The period was set long enough so that we can capture the rising and settling behavior of the actuator (motor driver) response (Fig.8).

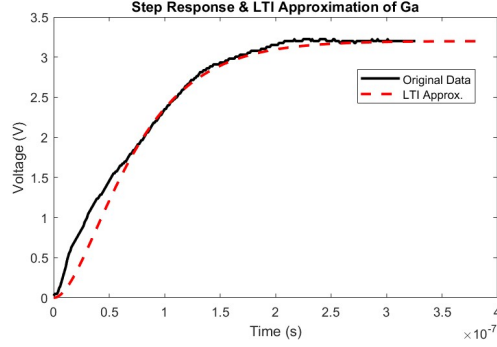


Fig. 8. Oscilloscope data showing the step response of G_a and its LTI approximation.

The LTI approximation for G_a (Fig. 8) is accomplished using the modified rise-time method as mentioned in [6] and shown in Eq. (13).

$$\frac{2.202 \times 10^{15}}{s^2 + 5.243 \times 10^7 s + 6.882 \times 10^{14}} \quad (13)$$

2) *Experimentally Determining Y_{elec}* : Electrical admittance can be described in the general LTI form [7] shown in Eq. 14:

$$Y_{elec} = \frac{1}{sL_w + R_w} \quad (14)$$

There are two desired values: R_w , and L_w (winding resistance and inductance of the motor, respectively). R_w was simply determined using an ohmmeter, which was found to be $R_w = 7.4\Omega$. To measure L_w , we constructed the experiment set-up as laid out in Fig. 10. By adding test resistor R_t in series with the motor, we measured the voltage drop transient across R_t in response to a square wave input (Fig. 10).

Fig. 10's decay time constant was measured to be $\tau = 1.52 \times 10^{-7}$ seconds. Using Eq. (15), we found L_w to be $2.716 \mu\text{H}$.

$$\tau = \frac{L_w}{R_{total}} = \frac{L_w}{R_w + R_t} \quad (15)$$

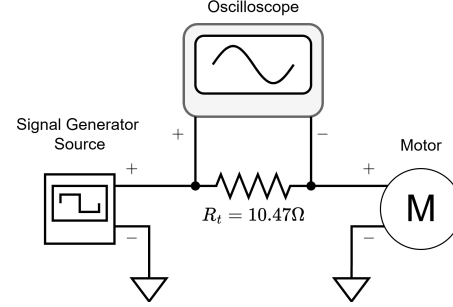


Fig. 9. Experiment setup for determining L_w .

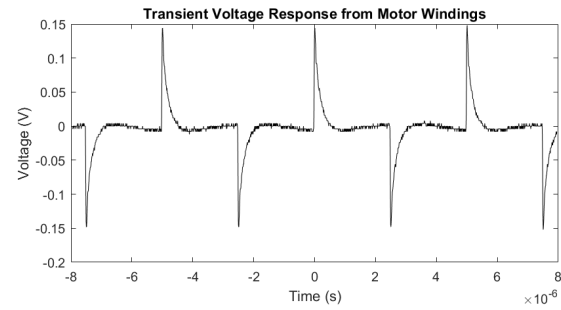


Fig. 10. Transient time plot of motor windings for analyzing the decay time constant τ .

F. Simulink Modeling

The Simulink model is shown in (Fig. 12). The PID values were found through methods of pole placement, just as taught in ELEC 341.

V. SOFTWARE DESIGN

A. IMU Pitch Angle Reading

To obtain the robot's tilt angle θ using the BMI270 IMU, we calculated the accelerometer tilt angle by $\theta_A = \arctan(\frac{-Ax}{\sqrt{Ay^2 + Az^2}})$, and the gyroscope tilt angle by $\theta_G = \theta_0 + Gy \cdot dt$. The accelerometer provided more accurate angle readings but it was more prone to noise, especially during linear acceleration, and the gyroscope outputted more constant and stable readings, but it had lots of drift. We experimented with using a complementary filter with a high gain between 0.95-0.98 to fuse the two sensors' data and remove linear acceleration noise while preventing gyroscope drift. However,

we decided to use a 1D Kalman filter due to its ability to dynamically bias the sensor data with less noise, which resulted in linear acceleration noise of 2° - 4° , while the complementary could not achieve noise level below 5° - 10° .

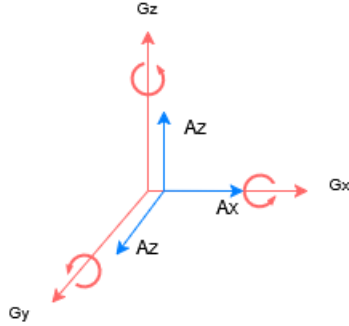


Fig. 11. Diagram of the IMU's accelerometer and gyroscope orientation.

The Arduino BMI270 library has a software frequency limit of 200Hz on the IMU readings, making it challenging to achieve responsive balance and control over the robot. The BMI270 supports up to 1.6kHz accelerometer frequency and 6.4kHz gyroscope frequency, therefore, we created our own custom BMI270 library with lower-level control over the I2C read and write commands between the Arduino and chip. With this library, we could disable the IMU's power saving mode and manually set both the accelerometer and gyroscope readings to 1.6kHz. This library significantly increased our robot's PID frequency from under 100Hz to over 800Hz, drastically improving the robot's responsiveness when balancing.

B. Magnetic Encoders

The magnetic encoders provide angular position readings of each wheel through I2C (using the wire library), allowing us to calculate the robot's displacement and yaw angle (its direction), which play a crucial role in the cascaded PID system. During calibration, the initial angle read by the encoders is recorded, and the change in angles with respect to the initial angle is used to calculate

the number of full rotations of the wheel as well as partial rotation as an angle. For instance, a change in angle from quadrant 4 to 1 increments the number of rotations, and a change in angle from quadrant 1 to 4 decrements the number of rotations. For displacement, we use the arc length formula, $\text{Displacement} = (2\pi \cdot N + \theta) \cdot R$, where N is the number of full rotations, θ is the partial angle in radians, and R is the radius of the robot's wheels. For yaw, we apply the small angle approximation for sine (calculated in radians), $\theta = \frac{(D_R - D_L)}{W}$, where D is the displacement of the wheel and W is the distance between the wheels. These calculations are essential for using the encoders in the PID to control the speed, positions and the direction of the robot.

C. PID Loop

The PID loop takes the current readings from the IMU and the encoders, and outputs a value between 0-255. Due to the dependency of a PID control system on derivative and integral calculations, it is important to keep precise track of time, which we did using the non-blocking micros function. The integral and derivative calculations for each cascaded PID subsystem were done by keeping track of the change in time and the previous sensor readings. The output of the PID system was directly used as a PWM value to drive each motor.

D. GUI and Bluetooth Communication

To control and monitor the robot from a computer, we created a graphical user interface using costumtkinter. We originally used PyQt, due to the faster processing times, however we eventually transferred the code to TKinter due to its ease of use when adding new components to the GUI and iterating different design ideas. TKinter text and button widgets allowed us to easily build a dashboard for displaying live telemetry data, adjusting the PID gains, and controlling/moving the robot.

The python section of the software is separated into a dashboard module with the core of the user interface, handling user input and streaming video, while the Bluetooth module manages asynchronous communication with the Arduino Nano.

We structured the BLE communication into a client generic attribute profile (GATT) representing the python bucktooth module and a server GATT for the Arduino. The Arduino server GATT consisted of a BLE service with a data out characteristic for receiving control data (forward and backward speed, direction), and a data in characteristic for sending telemetry data (PID loop calculations and sensor data). The actual data being sent between the Arduino and Python software was ASCII encoded strings and byte arrays.

E. Extra Features: ESP32 Cam and Steering Wheel

The ESP32 Cam module was programmed to serve as a TCP server and use the HTTP /stream request to stream video to the Python GUI. Upon receiving the /stream request from a client, the ESP32 Cam begins capturing images and compressing them to JPEG frames which are streamed using multipart HTTP responses. The client (Python GUI) parses each frame and displays it in sequence using a TKinter frame widget, simulating a video feed.

For the motion sensing steering wheel, a high gain complementary filter was used to fuse the x-axis accelerometer and gyroscope angles, which accurately recorded the rotation of the steering wheel (controlled the direction of the robot). The 4 switches and push buttons on the steering wheel were setup using builtin pull up resistors on the Arduino, and a debounce with 50ms delay ensured responsive noise free button inputs. All the steering wheel inputs were sent to the Python GUI over UART, allowing the user to control the robot with the wheel instead of the keyboard. On the python dashboard, the steering wheel angle set the direction of the robot, the push buttons moved the robot forward and backwards, and the paddle shifter switches incremented and decremented gears from 1-6. Our gear system was designed to create a more responsive driving experience by reducing the movement speed and increasing the rotation speed and lower gears, and having a higher movement speed and lower rotation speed at higher gears.

VI. VERIFICATION AND VALIDATION

A. Simulation Results

Using the PID values acquired using pole-placement, Fig. 13 shows the step-response produced by the $x-\theta$ sub-system.

B. Real-World Results

The final prototype of the robot passed all the functional and performance requirements. As highlighted in the demo video, the robot was able to recover from an angle of 28° and then balance vertically with a drift oscillation amplitude of about 2.5cm (required 15° disturbance and less than 10cm drift). For speed and direction control, using the software gear system we were able to achieve a rotation speed of $180^\circ/\text{s}$ at gear 1, and a movement speed of 0.45m/s at gear 6. As for the extra feature requirements, we were able to complete a figure 8 around obstacles using just the video stream and the steering wheel, highlighting that the video is stabilized, has minimal lag, and proving the responsiveness of the steering wheel for accurately controlling the robot in challenging environment.

VII. CONCLUSION

A. Fulfillment of Goals

As discussed in the verification and validation section, we not only passed all the general requirements, but also outperformed on all the quantitative performance requirements, showcasing that we were able to design a successful prototype for the self balancing inverted pendulum robot with cascaded PID. We also met the project's cost and time constraints while adhering to the main components provided in the robot kit. Throughout this project, we gained valuable experience in controls research, modeling and simulation of a real world physical system. We had to repeatedly improve our system's simulation model to achieve accurate results that would balance the physical robot. We also gained hardware and software design/debugging skills, which were essential to building the final prototype.

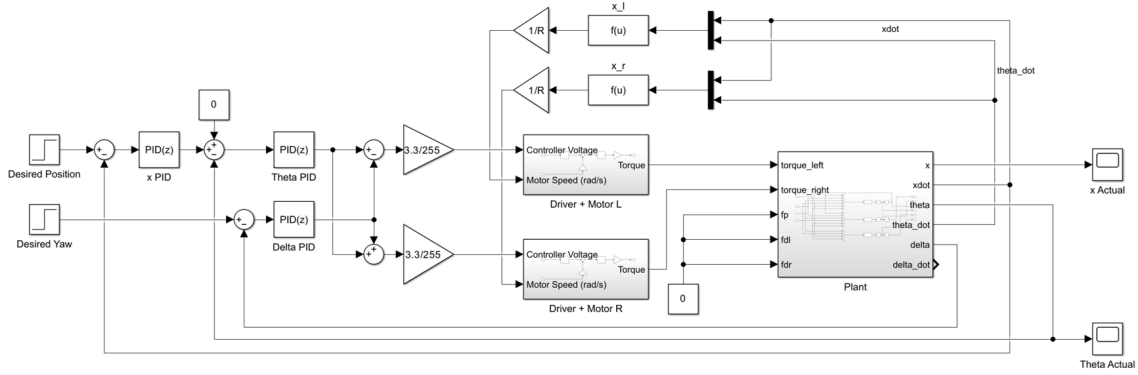


Fig. 12. The Cascaded PID Control System Diagram Designed in Simulink.

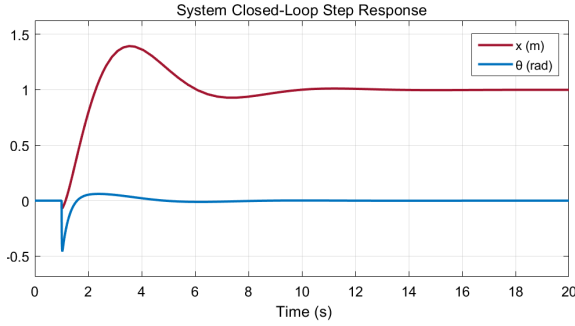


Fig. 13. Step-plot of the closed-loop x - θ sub-system.

B. Future Recommendations

- Improved battery pack: 4S(14.8V) Lipo battery pack with high mAh(+5000mAh) for longer use time and faster charge time using a dedicated battery management system with cell balancing.
- Back EMF: use brake resistors and flyback diodes to protect the power source and Arduino from sudden surges of current. This would provide the back EMF current a path to travel and dissipate without damaging the power source and electronics.
- PCB: design a custom PCB for all the electronic components to improve cable management and organization.

TABLE IV
BILL OF MATERIALS

Item	Price(CAD \$)
ESP32 Cam	\$23
MG90S 9G Servo Motor	\$3.6
Buck Boost Converter (x2)	2*\$3.2 = \$6.4
MPU6050 IMU	\$5.3
ELEGOO UNO R3 Board ATmega328P	\$22
	Total = \$60.3

- Bipedal wheeled balancing robot: experiment with a bipedal balancing robot with adjustable leg height using servo motors on each wheel. This could allow us to further improve the balance and recovery by dynamically adjusting the center of mass which could act as a damping mechanism on uneven terrain (or disturbances) or when accelerating/decelerating.

APPENDIX A BUDGET

APPENDIX B TECHNICAL CALCULATIONS AND SIMULATIONS

A. Non-linear Systems Equations Derivation using Lagrangian Mechanics

The kinetic energies of the system exists in both the base and pendulum. We will represent the

external force applied to the base as F . For the kinetic energy of the cart:

$$T_{\text{cart}} = \frac{1}{2}M\dot{x}^2$$

Designating x_p as the pendulum's position relative to ground, we can derive its velocity components relative to the cart. The pendulum's cartesian position:

$$x_p = x + l \sin \theta \quad y_p = l \cos \theta$$

And hence, its velocity:

$$\dot{x}_p = \dot{x} + l\dot{\theta} \cos(\theta) \quad \dot{y}_p = -l\dot{\theta} \sin(\theta)$$

The pendulum's kinetic energy can then be described as:

$$\begin{aligned} T_{\text{pendulum}} &= \frac{1}{2}m \left[(\dot{x} + l\dot{\theta} \cos \theta)^2 + (-l\dot{\theta} \sin \theta)^2 \right] \\ &= \frac{1}{2}m \left[\dot{x}^2 + 2l\dot{x}\dot{\theta} \cos \theta + l^2\dot{\theta}^2 \right] \end{aligned}$$

The total kinetic energy is therefore:

$$T = \frac{1}{2}(M + m)\dot{x}^2 + ml\dot{x}\dot{\theta} \cos \theta + \frac{1}{2}ml^2\dot{\theta}^2$$

Since we have set the wheel axle to a height of 0 ($y = 0$), the potential energy is simply:

$$V = -mgl \cos \theta$$

Solving the Euler-Lagrange equations for x :

$$\begin{aligned} \frac{\partial L}{\partial \dot{x}} &= (M + m)\dot{x} + ml\dot{\theta} \cos \theta \\ \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{x}} \right) &= (M + m)\ddot{x} - ml\dot{\theta}^2 \sin \theta + ml\ddot{\theta} \cos \theta \\ \frac{\partial L}{\partial x} &= 0 \end{aligned}$$

$$(M + m)\ddot{x} - ml\dot{\theta}^2 \sin \theta + ml\ddot{\theta} \cos \theta = F \quad (16)$$

And for θ :

$$\begin{aligned} \frac{\partial L}{\partial \dot{\theta}} &= ml\dot{x} \cos \theta + ml^2\dot{\theta} \\ \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) &= -ml\dot{x}\dot{\theta} \sin \theta + ml\ddot{x} \cos \theta + ml^2\ddot{\theta} \\ \frac{\partial L}{\partial \theta} &= -ml\dot{x}\dot{\theta} \sin \theta - mgl \sin \theta \\ \Rightarrow &\left[-ml\dot{x}\dot{\theta} \sin \theta + ml\ddot{x} \cos \theta + ml^2\ddot{\theta} \right] - \\ &\left[-ml\dot{x}\dot{\theta} \sin \theta - mgl \sin \theta \right] = 0 \end{aligned}$$

$$\ddot{x} \cos \theta + l\ddot{\theta} + g \sin \theta = 0 \quad (17)$$

B. Derivation of the Linear State-Space System

In order to model as an LTI (linear time-invariant) system, small angle approximations can be applied equations (16) and (17) where:

$$\sin \theta \approx \theta \quad \cos \theta \approx 1 \quad \dot{\theta} \approx 0$$

Which produces the following linearized equations (18), (19):

$$(M + m)\ddot{x} + ml\ddot{\theta} = F \quad (18)$$

$$\ddot{x} + l\ddot{\theta} + g\theta = 0 \quad (19)$$

By arranging (18) and (19) into matrix form, we can easily isolate for $\ddot{x}$ and $\ddot{\theta}$ allowing us to find the state-space matrices.

$$\begin{aligned} \begin{bmatrix} M + m & ml \\ 1 & l \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} &= \begin{bmatrix} F \\ -g\theta \end{bmatrix} \\ \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} &= \begin{bmatrix} M + m & ml \\ 1 & l \end{bmatrix}^{-1} \begin{bmatrix} F \\ -g\theta \end{bmatrix} \end{aligned}$$

$$\ddot{x} = \frac{F}{M} + \frac{mg}{M}\theta \quad (20)$$

$$\ddot{\theta} = -\frac{F}{Ml} - \frac{(M + m)g}{Ml}\theta \quad (21)$$

Since we have defined our state vector to be $\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T$, the state space system in the form of $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ is therefore:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{mg}{M} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -\frac{(M + m)g}{Ml} & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ \frac{1}{M} \\ 0 \\ -\frac{1}{Ml} \end{bmatrix}$$

APPENDIX C SCHEMATICS

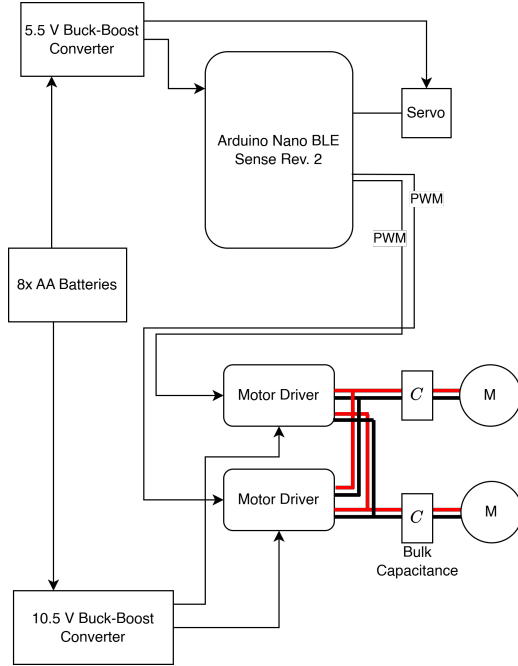


Fig. 14. Top-level schematic diagram.

APPENDIX D DATASHEET AND URLS FOR SELECTED COMPONENTS

- ESP32 Cam: URL = <https://www.universal-solder.ca/product/esp32-cam-with-esp32-cam-mb-usb-programming-adapter/>

APPENDIX E PROTOTYPES

Please see Fig. 15 in Appendix G for the initial “prototype” built from the original robot kit.

APPENDIX F SAMPLE CODE

**PLEASE SEE THE GITHUB REPOSITORY
FOR THE CODE**

<https://github.com/Mazda-F/391Robot.git>

A. Python Functions to find Moment of Inertia

```
import numpy as np
def gtest(data : dict):
    Tswing = data['Tswing']
    h = data['height']
    omega = 2*np.pi/Tswing
    gcalc = omega**2 * h
    greal = 9.81
    error_p = np.abs((greal - gcalc)/greal)
    ↪*100
    print("\nTesting_for_data_integrity]")
    print(f"Gravity:_{gcalc:.5g}m/s^2._
    ↪Error:{error_p:.5g}%\n")

def calcmoment(data : dict):
    g = 9.81
    m = data['mass']
    d = data['distance']
    h = data['height']
    Trot1 = data['Trot']/10.0
    inertia = m*g*(d**2) * (Trot1**2)/(16*(
    ↪np.pi**2) * h)
    print("\n[Moment_of_Inertia]")
    print(f"{inertia:.5g}\n")
```

APPENDIX G ADDITIONAL FIGURES

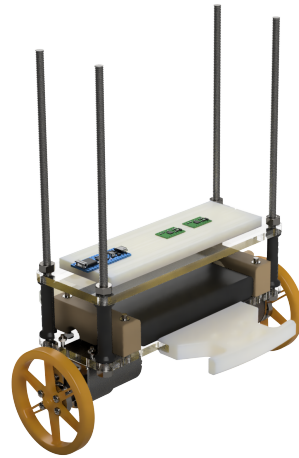


Fig. 15. CAD model rendering of the original robot kit.

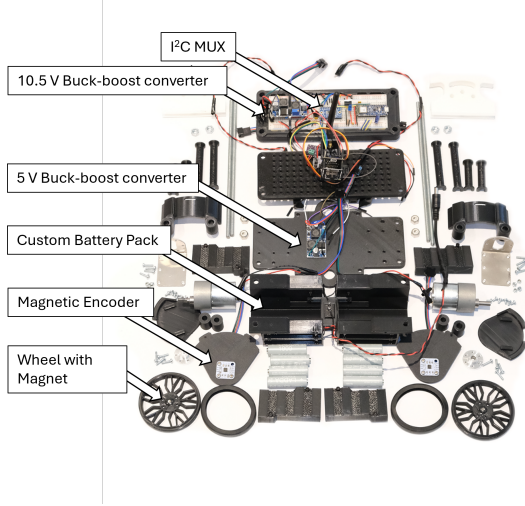


Fig. 16. Deconstructed robot displaying all the physical components.

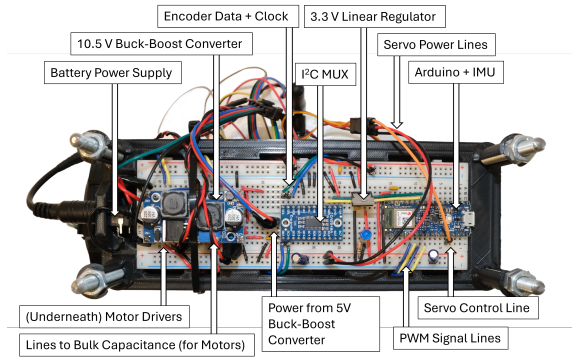


Fig. 17. Top-down view of the breadboard components and wiring.

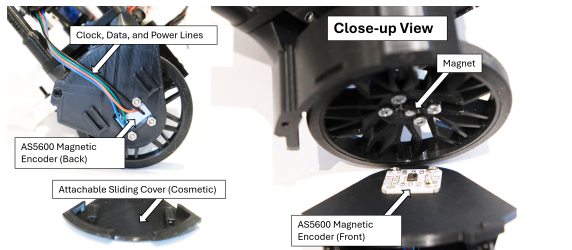


Fig. 18. Mounting mechanism for the AS5600 encoders.

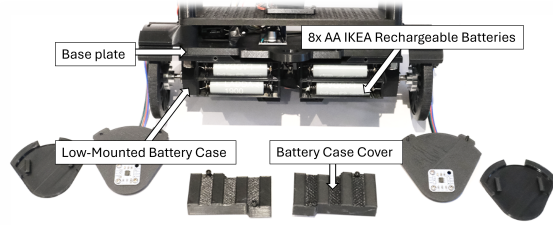


Fig. 19. The custom, low-mounted battery case.

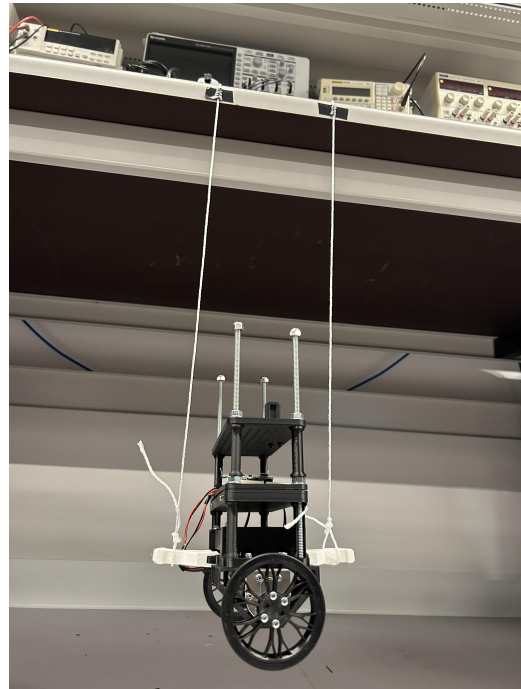


Fig. 20. Experimental setup for determining J_δ (the moment of inertia for yaw).

REFERENCES

- [1] F. Grasser, A. D'Arrigo, S. Colombi, and A. C. Rufer, "JOE: A Mobile, Inverted Pendulum," *IEEE Transactions on Industrial Electronics*, vol. 49, no. 1, pp. 107–114, Feb 2002.
- [2] B. Ata and R. Coban, "Linear quadratic optimal control of an inverted pendulum on a cart using artificial bee colony algorithm: An experimental study," *Çukurova University Journal of the Faculty of Engineering and Architecture*, vol. 32, pp. 109–124, 06 2017.

- [3] T.-J. Ren, T.-C. Chen, and C.-J. Chen, "Motion control for a two-wheeled vehicle using a self-tuning pid controller," *Control Engineering Practice*, vol. 16, no. 3, pp. 365–375, 2008. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0967066107001153>
- [4] H. A. Soule and M. P. Miller, "The experimental determination of the moments of inertia of airplanes," National Advisory Committee for Aeronautics, Technical Report NACA-TR-467, 1934, document ID: 19930091541; Accession Number: 93R20831; Date Acquired: September 6, 2013; Acquisition Source: Legacy CDMS; Public Distribution; Copyright: Work of the US Gov. Public Use Permitted.
- [5] M. Koken, "The experimental determination of the moment of inertia of a model airplane," 2017, honors Research Projects. 585.
- [6] L. Stocco, "Optimal 2nd order lti system identification," in *2023 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 2023, pp. 960–965.
- [7] —, "Elec 341: Motor model [pdf slides]," 2024.